보안 진단 · 웹 애플리케이션

실습용 웹 애플리케이션 취약점 진단 및 개선

직접 구성한 격리 환경에서 취약점을 재현하고, 원인 분석·패치·재검증까지 단독으로 수행했습니다.

발견 **7건** | 수정 **7건** | 재검증 **완료** | 취약 유형 **5개**

수행 역할	실습 환경 구성 · 취약점 진단 · 코드 패치 · 재검증 (단독 수행)
진단 대상	로그인 · 상품 검색/상세 · 상품 후기 · 주문 조회 · 로그인 세션
환경	Python 3 (http.server·sqlite3) · SQLite · CentOS 9 · VMware 사설망(외부 비공개)
코드·실행	일부러 취약하게 구성한 실습용 서버 — 상세 코드와 실행 방법은 요청 시 제공(비공개)

※ 각 항목의 재검증 결과는 "안전함"을 보장하는 것이 아니라, 해당 실습 시나리오에서 공격이 재현되지 않음을 확인한 것입니다.

한눈에 보기

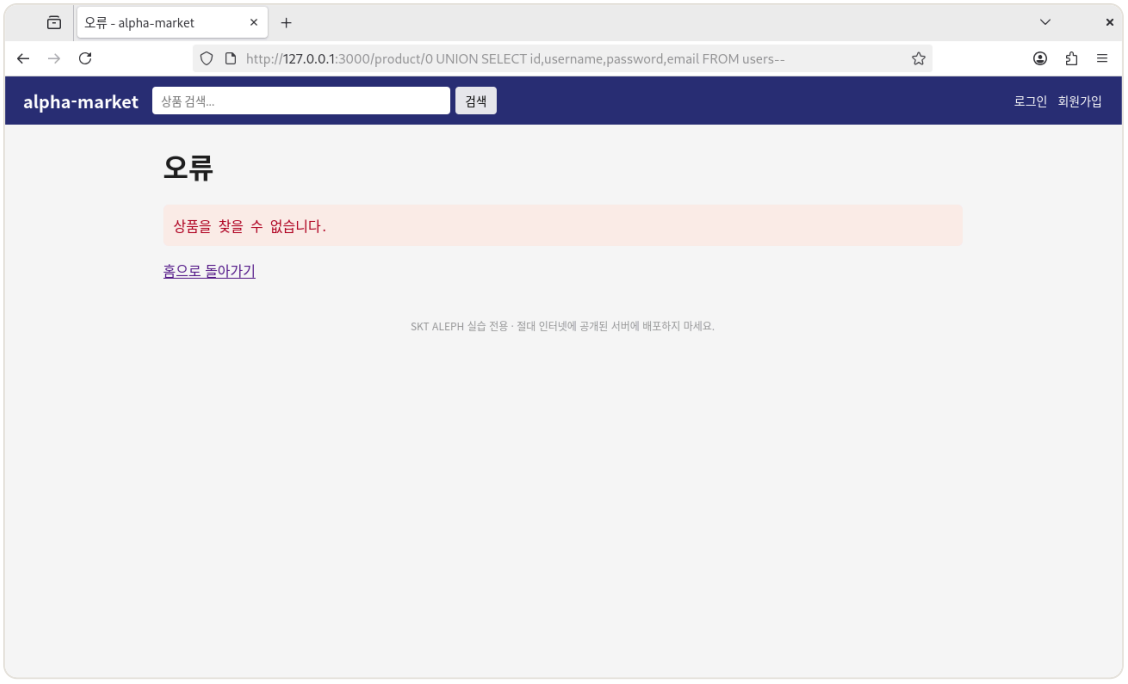
1	인젝션	SQL 인젝션 (로그인·검색·상품상세 3곳)	파라미터 바인딩
2	인젝션	저장형 XSS (상품 후기)	출력 이스케이프
3	데이터	비밀번호 평문 저장	해시 + 솔트
4	접근제어	IDOR (주문 조회 권한 우회)	소유자 검증
5	인증·세션	세션 쿠키 위조	서명 검증 + HttpOnly

상세 진단

01 SQL 인젝션

인젝션

대상·영향	로그인 폼·상품 검색·상품 상세. 인증 우회나 타 계정 정보 열람으로 이어질 수 있었음.
재현 조건	로그인·검색·상세 기능에서 입력란·주소 파라미터에 SQL 구문이 섞인 값을 넣어 쿼리 구조를 바꾸는 요청.
취약 원인	취약 입력값을 쿼리 문자열에 그대로 이어 붙여, 입력이 코드로 해석됨.
조치	조치 로그인·검색·상품상세 3곳 모두 파라미터 바인딩으로 값과 쿼리를 분리. `conn.execute(sql, (값,)) # ? 로 바인딩`
재검증	통과 조작된 주소로 접근해도 정보가 노출되지 않고 오류로 차단, 정상 조회는 유지.



상품상세에 UNION 구문을 넣어 접근 시 "상품을 찾을 수 없습니다"로 차단

02 저장형 XSS

인젝션

대상·영향 상품 후기. 후기를 보는 모든 사용자의 브라우저에서 임의 스크립트가 실행될 수 있었음(세션 탈취 등).

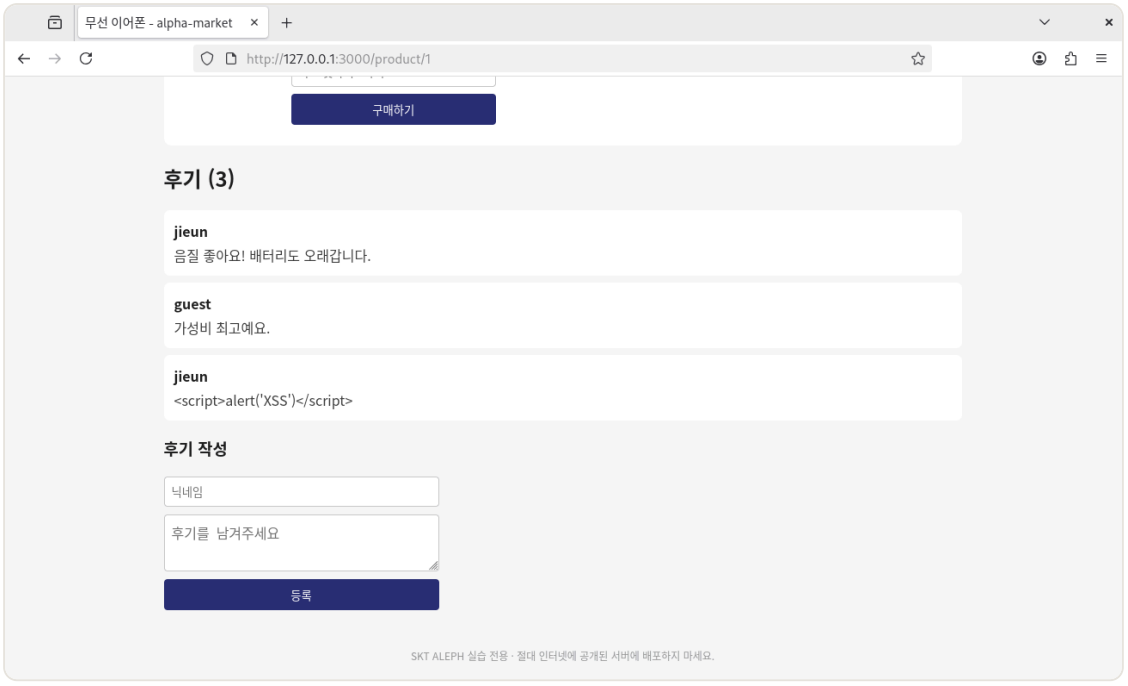
재현 조건 로그인한 사용자가 후기 입력란에 스크립트 태그가 포함된 내용을 등록한 뒤, 그 상품 페이지를 다른 사용자가 여는 상황.

취약 원인 취약 후기 내용을 이스케이프 없이 그대로 화면에 출력.

조치 조치 출력 시 특수문자를 이스케이프 처리.

```
{esc(r['content'])}
```

재검증 통과 삽입한 스크립트가 실행되지 않고 텍스트로만 표시됨.



후기에 넣은 <script> 태그가 실행되지 않고 글자 그대로 표시

03 비밀번호 평문 저장

데이터 보호

대상·영향	회원 DB. 유출 시 전 계정의 비밀번호가 즉시 노출될 수 있었음.
재현 조건	DB 파일(alpha_market.db)에서 users 테이블의 password 컬럼을 직접 확인.
취약 원인	취약 비밀번호를 암호화 없이 평문 그대로 저장.
조치	조치 PBKDF2 해시 + 사용자별 솔트로 저장, 로그인은 해시 비교로 변경. `hash_password(pw) 저장 · verify_password(입력, 해시)`
재검증	통과 DB에 원문 대신 해시값만 저장되어 원문 확인이 불가능.

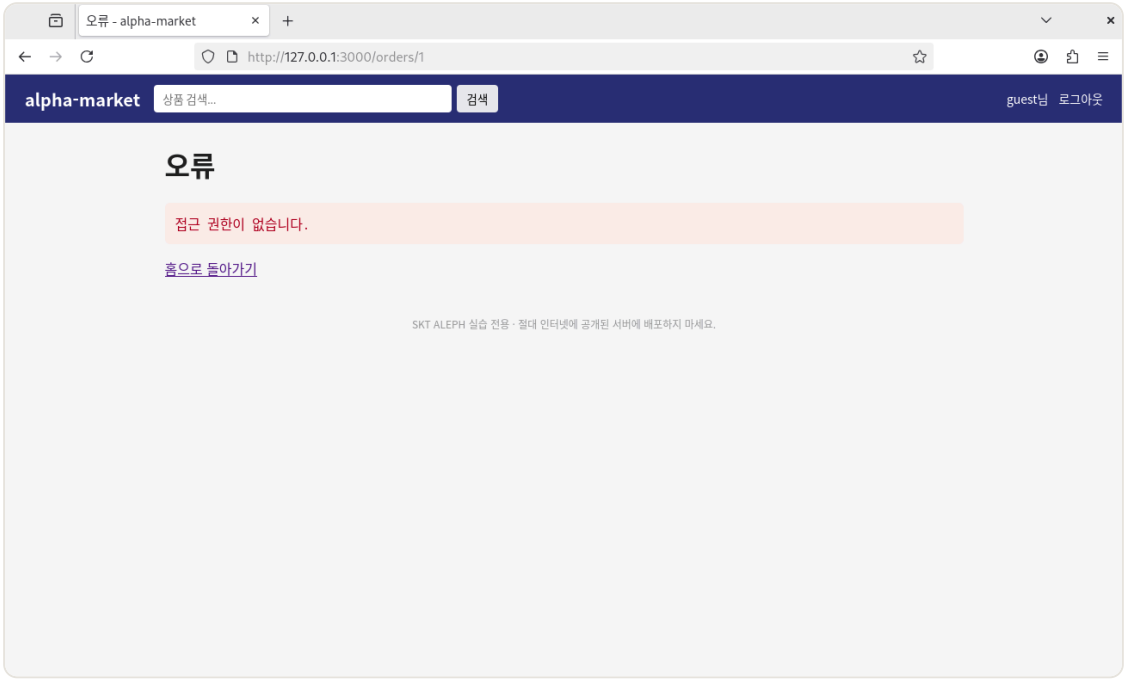
```
[root@localhost alpha-market-vuln]# sqlite3 alpha_market.db "SELECT username, password FROM users;"
admin|fd9bf85ba2a25a2dddc497190b23b9db$1cf25876fdd1b00affd017813174ca4a7e0c51c7042b8e5fcc96d6c8eba4fdef
jieun|4e9c075d204b04dd74d1830693bb14de$4a30223da20cbacc1c42cede867638b7353878da1d242192756f4dae6f5039db
guest|4afa215a01b134819495efff481d047f$ccc56cc26355cdbc7ecc79932488be5239efd630254c48fe59a778a716a885d6
```

DB 조회 결과 — 비밀번호가 "솔트$해시" 형태로 저장됨

04 IDOR — 주문 조회 권한 우회

접근 제어

대상·영향	주문 상세(/orders/{id}). 로그인만 하면 타인의 주문·배송지·결제정보를 열람할 수 있었음.
재현 조건	guest 계정으로 로그인한 상태에서 주소의 주문 번호를 본인(#4)이 아닌 번호(#1)로 바꿔 요청.
취약 원인	취약 주문을 번호로 조회한 뒤 소유자 일치 여부를 확인하지 않고 응답.
조치	조치 조회한 주문의 소유자와 로그인 사용자가 다르면 차단. `if order["user_id"] != user["id"]: 접근 차단`
재검증	통과 타인 주문 번호 접근 시 차단, 본인 주문은 정상 조회.

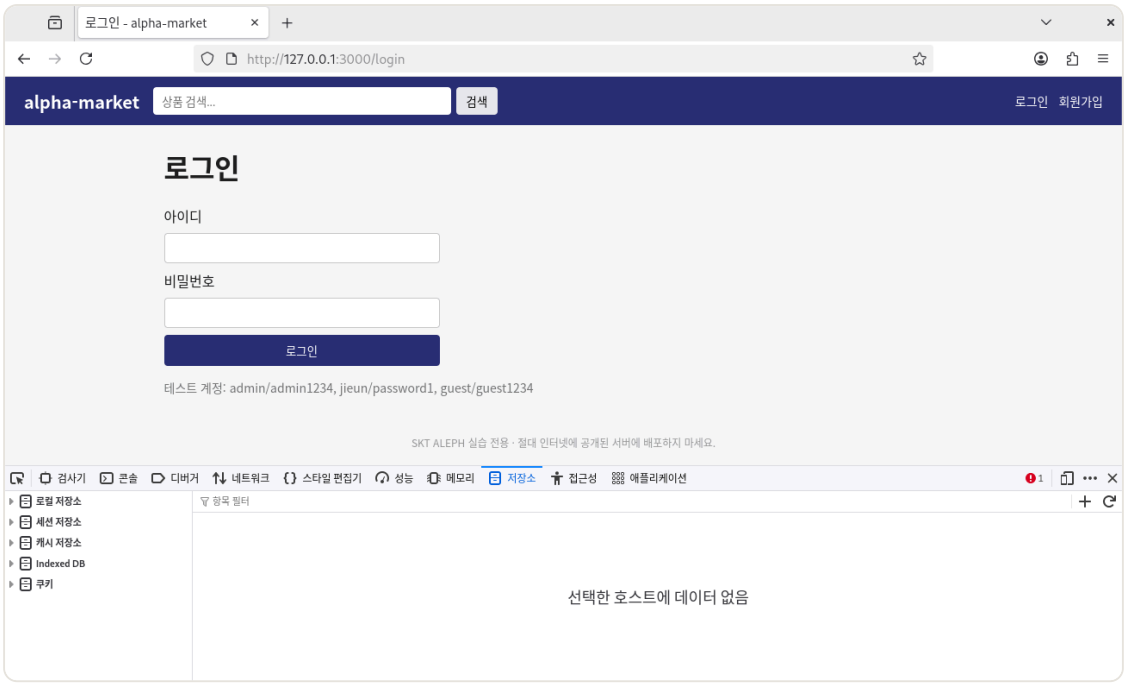


다른 사용자의 주문(/orders/1) 접근 시 "접근 권한이 없습니다"

05 세션 쿠키 위조

인증·세션

대상·영향	로그인 세션. 쿠키 값을 바꾸면 다른 계정, 심지어 관리자로 위장할 수 있었음.
재현 조건	로그인한 상태에서 개발자도구로 세션 쿠키 값을 admin으로 바꾼 뒤 다시 요청.
취약 원인	취약 쿠키에 아이디를 평문으로 담고, 읽을 때 값을 그대로 신뢰.
조치	조치 쿠키에 HMAC 서명을 붙이고 읽을 때 서명을 검증, HttpOnly 적용. `sign_value(username) 발급 · unsign_value()로 검증`
재검증	통과 쿠키를 admin으로 조작해도 서명 불일치로 거부되어 로그인이 해제됨.



쿠키를 admin으로 바꾼 뒤 새로고침 시 로그인 해제(로그인 화면으로 복귀)

배운 점

- 입력값을 신뢰하지 않고, DB 질의·화면 출력에서 **값과 코드를 분리**하는 것이 기본이라는 점
- 보안은 한 번에 막는 것이 아니라 **인증·권한·저장·세션**까지 여러 지점에 겹겹이 쌓아야 한다는 점
- 조치로 끝내지 않고 **공격은 막히고 정상 기능은 유지되는지 직접 재검증**해야 한다는 점

© 2026 전지은 · 실습은 외부와 격리된 VMware 환경에서만 진행했습니다.